



UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Parametric Polymorphism in C++, Java, and C#
Generics vs Templates and Metaprogramming
Power

Prompt log

Course:

Advanced Programming

Student:

Elia Leonardi

Chapter 1

AI Prompt Engineering

To ensure the technical accuracy of the generated content and its adherence to the project guidelines, this report was developed using a **two-phase prompting strategy**: prompt improvement and prompt execution. A fundamental principle of this methodology is strict context isolation, under which prompt refinement and content generation are treated as entirely separate operations, each carried out without knowledge of the other's context.

1.1 Architect Chat

An initial “Architect Chat”, run with Gemini Pro 3.1 Extended, was used to plan and outline the optimal sequence of prompts, ensuring that all project requirements were systematically addressed before any content generation began. This planning stage produced the list of prompts later refined and executed in the two phases described below, but did not itself generate any content included in the report.

1.2 Prompt Engineering Phases

1.2.1 Prompt Improvement

The first phase focuses on prompt refinement. During this stage, the initial prompts produced by the Architect Chat are iteratively rewritten to align with the project guidelines and to maintain a rigorous academic standard. Each prompt is refined independently: the AI model is tasked solely with improving the prompt's structure and clarity, without executing it and without carrying over context from unrelated topics or previous prompts.

1.2.2 Prompt Execution

The second phase involves executing the refined prompts to generate the final content, using Claude Sonnet 5 in a new, isolated session for each prompt. This isolation ensures that the generated content relies exclusively on the explicit instructions contained in the finalized prompt, without any context leakage from the improvement phase or from other prompts. During execution, the model is also explicitly instructed to report the source references and documentation used to formulate its response.

1.3 Prompt 0: Configure Prompt Engineering (Task 1)

To ensure the generation of rigorous academic sections, this methodology treats prompt generation and the final report generation as strictly independent operations. The process does not rely on manual, ad-hoc prompts for each language. Instead, it employs a *meta-prompting* approach. A **Master Prompt** was designed whose sole purpose is to act as a

"Senior Prompt Engineer" in an isolated session. This system takes a raw specification as input and outputs a formal, optimized execution prompt, which is then used in a completely separate session to generate the final text.

Master Prompt Specification

Below is the complete text of the Master Prompt used to orchestrate the prompt engineering phase for Task 1 (language implementation of parametric polymorphism):

System Persona:

Act as a Senior Prompt Engineer specialized in designing rigorous prompts for the generation of technical-academic reports in computer science, programming language theory, compiler design, and software engineering.

Goal:

Given, as input, a task specification containing a persona, goal, requirements, writing-style constraints, output structure, and possible reference requirements for producing a technical-academic report, generate a single, self-contained master prompt ready to be executed in a new language model conversation.

The generated master prompt must preserve the original intent and must be optimized to produce a formally structured academic report with appropriate technical depth, logical organization, and scientific writing standards.

Requirements:

Task 1, Extraction and Preservation:

Parse the input specification and identify:

- The requested system persona or expert role.
- The report objective and title, if provided.
- All explicit and implicit technical requirements.
- Required theoretical concepts, comparisons, analyses, and evaluations.
- Writing-style constraints.
- Required output format and document structure.
- Requirements related to references, citations, bibliography, or external academic sources.

Preserve the complete meaning of the original specification.

Do not remove requirements.

Do not introduce new technical requirements.

Do not change the intended scope of the report.

Do not simplify technical constraints.

Task 2, Correction and Structuring:

Correct grammatical errors, ambiguities, inconsistent terminology, formatting problems, and unclear instructions present in the input.

Reorganize the extracted information into a clear and logically structured master prompt using the following sections:

- System Persona
- Goal
- Requirements

- Writing Style Constraint
- Output Structure

When reference requirements are present, preserve them explicitly in the Requirements section.

Task 3, Academic Report Generation Constraints:

Ensure that the generated master prompt instructs the target language model to:

- Produce a complete technical-academic report rather than a short summary.
- Maintain a formal, objective, and scientific writing style.
- Use precise terminology appropriate for computer science literature.
- Provide a logical progression from theoretical foundations to technical analysis.
- Include comparisons, trade-offs, and implementation details when required.
- Distinguish clearly between theoretical concepts and implementation mechanisms.
- Avoid unsupported claims and clearly identify assumptions.
- ENFORCE STRICT SCOPE CONTROL: Explicitly instruct the target language model to confine the generated text strictly to the requested core topic. The model must actively avoid tangential subjects (e.g., memory footprint, hardware performance, code bloat, garbage collection overhead, or broader language comparisons) unless they are explicitly demanded by the input specification.

Task 4, Reference and Citation Requirements:

If the original specification requires the use of references:

- Require the generated report to rely on authoritative academic or technical sources.
- Prefer peer-reviewed publications, academic books, official language specifications, and authoritative technical documentation.
- Require consistent citation formatting.
- Require a final bibliography section containing all cited resources.
- Avoid fabricated references, unverifiable sources, or unsupported citations.

Task 5, LaTeX Output Requirements:

If the original specification requires LaTeX output:

- Require complete compilation-ready LaTeX code.
- Preserve a professional academic document structure.
- Include appropriate sections, chapters, mathematical notation, tables, figures, citations, and bibliography commands when required.
- Ensure compatibility with standard LaTeX compilation workflows.

Writing Style Constraint (STRICT):

Do not execute the input specification.
Do not generate the final report.
Do not answer the original technical question.

The output must contain only the generated master prompt.

Do not include:

- explanations,
- comments,
- editorial notes,
- placeholders,
- analysis of the transformation,
- meta-commentary addressed to the user.

Do not use first-person forms ("I", "we") in the generated master prompt unless they are explicitly required by the original specification.

Output Structure:

Return only the generated master prompt.

The generated prompt must be formatted as ready-to-use text and organized using the following headings:

System Persona

Goal

Requirements

Writing Style Constraint

Output Structure

No additional text must appear before or after the generated master prompt.

1.4 Prompt Obis: Configure Prompt Engineering (Task 2)

For Task 2 (metaprogramming across C++, Java, and C#), a second Master Prompt was used. Its purpose is identical to the one described above, but it is specialized to produce a single *section* of an existing chapter (rather than a standalone report), given one focused instruction as input.

System Persona:

Act as a Senior Prompt Engineer specialized in designing rigorous prompts for the generation of individual sections within a larger technical-academic report chapter on programming language theory and software engineering.

Goal:

Given, as input, a single focused instruction (below, marked as [INPUT PROMPT]), generate a single, self-contained master prompt ready to be executed in a new language model conversation. The master prompt must instruct the target language model to produce ONE SECTION of a larger chapter -- not a standalone report -- on parametric polymorphism and metaprogramming across C++, Java, and C#.

[INPUT PROMPT]:
"{{PASTE HERE ONE OF THE 9 PROMPTS}}"

Requirements:

Task 1 -- Extraction and Preservation:

Parse [INPUT PROMPT] and identify:

- The specific technical question or comparison it asks for.
- The languages, techniques, or mechanisms explicitly named.
- Any implicit expectation (e.g. code examples, explanation of underlying mechanism, comparison table).

Preserve the complete meaning of [INPUT PROMPT]. Do not remove requirements. Do not introduce new technical requirements beyond what [INPUT PROMPT] asks. Do not expand the scope beyond the specific question posed.

Task 2 -- Section-Level Framing (STRICT):

The generated master prompt must explicitly instruct the target language model that its output is ONE SECTION of an existing chapter, not a complete report. Specifically, the master prompt must require the target model to:

- Omit any standalone introduction, motivation, or "in this report we will" framing -- begin directly with the technical content.
- Omit any standalone conclusion, summary of the whole chapter, or forward-looking remarks about other sections.
- Omit its own bibliography or reference list -- citations, if any, are consolidated once at the end of the full chapter, not per section.
- Assume the reader already has the context of the broader comparison (C++ templates vs Java generics vs C# generics) and does not need it re-introduced.

Task 3 -- Content Constraints:

Ensure the generated master prompt instructs the target language model to:

- Maintain a formal, objective, and precise technical writing style consistent with programming language theory literature.
- Use precise terminology (instantiation, monomorphization, type erasure, reification, substitution failure, bounded polymorphism) only where relevant to [INPUT PROMPT].
- Include working code examples only if [INPUT PROMPT] explicitly or implicitly requires them.
- Distinguish clearly between compile-time and runtime mechanisms when relevant to the specific question.
- Avoid unsupported claims; state assumptions about language/compiler version only if relevant to the specific question asked.
- ENFORCE STRICT SCOPE CONTROL: address only what [INPUT PROMPT] asks. Do not digress into memory footprint, hardware performance benchmarking, code bloat, garbage collection overhead, or unrelated language comparisons unless [INPUT PROMPT] explicitly requires it.
- ENFORCE CONCISENESS: the section must be as short as fully answering [INPUT PROMPT] allows. No padding, no restating the question, no redundant transitions between paragraphs.

Writing Style Constraint (STRICT):

Do not execute [INPUT PROMPT] directly. Do not generate the final section content. Do not answer the original technical question. The output must contain only the generated master prompt. Do not include explanations, comments, editorial notes, placeholders, analysis of the transformation, or meta-commentary addressed to the user. Do not use first-person forms ("I", "we") in the generated master prompt unless explicitly required by [INPUT PROMPT].

Output Structure:

Return only the generated master prompt, formatted as ready-to-use text and organized using the following headings: System Persona, Goal, Requirements, Writing Style Constraint, Output Structure. No additional text must appear before or after the generated master prompt.

Chapter 2

Task 1 – Language Implementation of Parametric Polymorphism

This chapter groups the prompts used to generate the report content addressing Task 1: *“Explain how each language implements parametric polymorphism (C++ templates, Java generics, C# generics), including when instantiation happens, what exists at runtime, and how this affects performance, type safety, and reflection.”* Each section below merges what were previously three separate, per-language prompts (C++, Java, C#) into a single comparative prompt covering all three languages together, so that a single generation directly produces the cross-language comparison.

Methodological Note on Prompt Construction and Verification:

To ensure the accuracy of the generated analysis, the prompt engineering strategy utilized a decoupled approach. Specifically, individual prompts were tailored for each language (C++, Java, and C#) across five core analytical categories:

1. Timing of instantiation
2. Runtime representation
3. Performance implications
4. Type safety
5. Reflection capabilities

Because multiple distinct queries targeted different facets of the same language, the model’s outputs naturally contained redundant technical information. This redundancy was intentional; it served as the critical first step in the verification process, allowing for the cross-referencing of the model’s overlapping responses to ensure internal consistency before synthesizing the final master prompt.

To simplify the analysis and improve readability within this document, these isolated queries were subsequently merged. The resulting unified master prompt dictates the exact structure, constraints, and content parameters used to generate the comprehensive comparison presented in the following sections.

Note on repeated boilerplate: from Prompt 3 onward, the **Writing Style Constraint** and **Output Structure** blocks follow the same template established in Prompt 2 (§2.2), adapted only in terminology and section titles to the specific topic. To avoid redundancy, these two blocks are stated in full only once (Prompt 2) and referenced thereafter.

2.1 Prompt 1: Overview of Parametric Polymorphism

Model: ChatGPT-5.5

Goal: Generate an overview of Parametric Polymorphism and ad-hoc polymorphism, giving details of type erasure compared with generics and monomorphization, and comparing differences between source code, bytecode, and binary code.

Prompt:

System Persona:

Act as a world-class computer scientist and academic researcher specialized in programming language theory, type systems, and the historical foundations of polymorphism.

Goal:

Produce an exhaustive, publication-grade theoretical overview of parametric polymorphism and ad-hoc polymorphism, comparing type erasure against reified generics and monomorphization, and explaining how these strategies manifest differently across source code, bytecode, and binary code.

Requirements:

- Provide an exhaustive theoretical background on parametric polymorphism.
- Define and thoroughly explain Christopher Strachey's distinction between parametric polymorphism and ad-hoc polymorphism.
- Detail the two primary implementation strategies discussed in the literature:
 1. Homogeneous translation (type erasure), detailing how it results in one compiled representation for all instantiations.
 2. Heterogeneous translation (reification/monomorphization), detailing how it results in separate code generated per instantiated type.
- Analyze type erasure versus reified generics as the core axis of comparison.
- Analyze compile-time versus runtime characteristics as the second axis of comparison, specifically examining when instantiation happens and what type information survives into the compiled binary or bytecode.

Writing Style Constraint:

Maintain a formal, objective, and precise academic writing style consistent with programming language theory literature. Avoid first-person pronouns. Provide technically detailed explanations rather than high-level summaries.

Output Structure:

The entire document must be written in professional, compilation-ready LaTeX code, organized into clearly labeled sections covering the theoretical background, the Strachey distinction, the two translation strategies, and the comparative analysis.

Sources:

- Strachey, “Fundamental Concepts in Programming Languages” [33] – used

for the original classification of parametric and ad-hoc polymorphism.

- **Cardelli and Wegner, “On Understanding Types, Data Abstraction, and Polymorphism”** [5] – used for the analysis of type abstraction and forms of polymorphism.
- **Pierce, *Types and Programming Languages*** [29] – used for the formal treatment of parametric polymorphism and type systems.
- **Pierce, *Advanced Topics in Types and Programming Languages*** [30] – used for advanced insights into type systems.
- **Kennedy and Syme, “Design and Implementation of Generics for the .NET CLR”** [17] – used for the description of runtime reification in the CLR.
- **Bracha et al., “Making the Future Safe for the Past”** [4] – used for the type erasure mechanism introduced in Java generics.
- **Bracha, *Generics in the Java Programming Language*** [3] – used for the technical documentation of Java erasure.
- **ECMA-334: C# Language Specification** [10] – used as a normative reference for C# generics.
- **ECMA-335: Common Language Infrastructure** [11] – used for the runtime representation of generics in the CLR.
- **The Java Language Specification** [26] – used as a normative reference for Java generics.
- **The Java Virtual Machine Specification** [27] – used for the bytecode representation resulting from erasure.
- **Cooper and Torczon, *Engineering a Compiler*** [6] – used for general compilation context.
- **Appel, *Modern Compiler Implementation*** [2] – used for general compilation context.
- **Aho et al., *Compilers: Principles, Techniques, and Tools*** [1] – used for compiler theory.
- **Official Rust documentation** [36] – used for a comparison on monomorphization in other languages.
- **Rust documentation on monomorphization** [35] – used to detail the monomorphization mechanism.
- **Vandevorde, Josuttis, Gregor, *C++ Templates: The Complete Guide*** [37] – used for the C++ template instantiation model.
- **Swift documentation on generics** [19] – used for an additional comparison on generics implementation strategies.

2.2 Prompt 2: Instantiation Time – C++, Java, and C# Compared

Goal: Generate a single, unified, technically rigorous section comparing the *instantiation time* of parametric polymorphism across C++ templates, Java generics, and C# generics, covering compile-time template instantiation, compile-time type erasure, and runtime CLR/JIT-based instantiation within one comparative treatment.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as a world-class computer scientist and principal compiler/runtime engineer specializing in Programming Language Theory (PLT), type systems, and the compiler/virtual-machine execution models of C++, the JVM, and the .NET CLR.

Goal:

Generate an exhaustive, publication-grade academic report section analyzing the instantiation time of parametric polymorphism, contrasting three distinct temporal models: C++ compile-time template instantiation, Java compile-time type erasure, and C# runtime CLR/JIT instantiation.

Requirements:

- C++ instantiation model: Explain the complete compile-time instantiation pipeline, including template argument deduction, two-phase name lookup, implicit and explicit template instantiation, and monomorphization/specialization. Emphasize that no template instantiation occurs during program execution. Describe how multiple translation units may independently instantiate identical specializations and how linkers eliminate duplicates (COMDAT sections, weak symbols).
- Java instantiation model: Explain the Java Generics compilation pipeline, including compile-time verification of generic constraints and bounds, type checking of parameterized types, type erasure transformation, replacement of type variables with erasures, generation of synthetic bridge methods, and insertion of implicit runtime casts in the generated bytecode. Emphasize that no generic instantiation occurs at runtime; only a single erased representation exists.
- C# instantiation model: Explain the dual-phase mechanism of C# generics: compile-time emission of Common Intermediate Language (CIL) with generic metadata preserved, followed by runtime Just-In-Time (JIT) instantiation. Detail reference-type code sharing versus value-type specialization performed by the JIT.
- Comparative synthesis: Directly compare the three timelines -- C++ ahead-of-time compile-time monomorphization, Java compile-time erasure with no runtime instantiation, and C# runtime JIT-time specialization -- making explicit when instantiation happens in each model and what triggers it.
- Whenever appropriate, relate each implementation strategy to the theoretical notion of type substitution in parametric polymorphism.
- ENFORCE STRICT SCOPE CONTROL: confine the text strictly to instantiation timing and mechanisms. Do not discuss executable size, code bloat, memory footprint, garbage collection overhead, or general runtime performance/speed.
- Base every technical statement on authoritative references, prioritizing the ISO/IEC 14882 C++ Standard, the Java Language Specification, the Java Virtual Machine Specification, ECMA-334, and ECMA-335.
- Include a complete bibliography using standard LaTeX thebibliography commands.

Writing Style Constraint:

- Adopt a formal, objective, and precise scientific writing style comparable to ACM or IEEE publications.
- Use accurate compiler/runtime terminology (e.g., template argument deduction, two-phase lookup, monomorphization, translation unit, COMDAT, type erasure, bridge method, JIT specialization, reification).
- Avoid first-person pronouns and conversational language.
- Provide technically detailed explanations rather than high-level summaries.

Output Structure:

The generated document must consist entirely of valid, self-contained, compilation-ready LaTeX code, organized as:

1. Introduction: Three Models of Instantiation Time
2. C++ Templates: Compile-Time Instantiation Pipeline
3. Java Generics: Compile-Time Type Erasure
4. C# Generics: Runtime CLR/JIT Instantiation
5. Comparative Timeline: When Instantiation Happens in Each Language
6. Conclusion

Sources:

- **ISO/IEC 14882:2020/2024, Programming Languages — C++** [15] – used as a normative reference for template instantiation.
- **Vandevoorde, Josuttis, Gregor, *C++ Templates: The Complete Guide*** [37] – used for the template deduction and instantiation pipeline.
- **Stroustrup, *The C++ Programming Language*** [34] – used for the general C++ compilation model.
- **The Java Language Specification, Java SE 24** [26] – used for the type erasure mechanism.
- **The Java Virtual Machine Specification, Java SE 24** [27] – used for the byte-code representation resulting from erasure.
- **Bracha et al., “Making the Future Safe for the Past”** [4] – used for the introduction of generics in Java.
- **Bracha, *Generics in the Java Programming Language*** [3] – used for the implementation details of erasure.
- **Igarashi, Pierce, Wadler, “Featherweight Java”** [13] – used for the formal model of Java generics.
- **ECMA-334: C# Language Specification** [10] – used as a normative reference for C# generics.
- **ECMA-335: Common Language Infrastructure** [11] – used for the JIT instantiation mechanism.
- **Kennedy and Syme, “Design and Implementation of Generics for the .NET CLR”** [17] – used for the JIT sharing/specialization mechanism.
- **Richter, *CLR via C#*** [32] – used for the runtime behavior of the CLR.
- **Pierce, *Types and Programming Languages*** [29] – used for the theoretical concept of type substitution.
- **Aho et al., *Compilers: Principles, Techniques, and Tools*** [1] – used for general compilation context.

- Cooper and Torczon, *Engineering a Compiler* [6] – used for general compilation context.

2.3 Prompt 3: Runtime Representation – C++, Java, and C# Compared

Goal: Generate a single, unified section analyzing the *runtime representation* of parametric polymorphism across C++ templates (monomorphized native code, no generic metadata), Java generics (erased representation), and C# generics (reified CLR representation).

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as a world-class computer scientist and principal compiler/runtime engineer specializing in Programming Language Theory (PLT), compiler backends (GCC/Clang), the Java Virtual Machine, and the .NET Common Language Runtime.

Goal:

Generate an exhaustive, publication-grade academic report section analyzing the instantiation time of parametric polymorphism, contrasting three distinct temporal models: C++ compile-time template instantiation, Java compile-time type erasure, and C# runtime CLR/JIT instantiation.

Requirements:

- C++: Explain heterogeneous translation (monomorphization) as producing distinct, independent native machine code artifacts per instantiated type. Clarify the absence of generic metadata at runtime: the execution environment sees only ordinary functions and objects, not templates. Address the limited type information that survives (name mangling, RTTI) and how this differs from reification.
- Java: Explain how type erasure produces a single shared runtime representation with no generic metadata for parameter types, the insertion of implicit casts, bridge methods, and the consequences for primitive types (boxing/unboxing).
- C#: Explain CLR reification: how generic metadata is preserved in CIL and how the JIT constructs runtime representations, distinguishing reference-type code sharing (with runtime dictionary passing) from value-type specialization (distinct native code per value type).
- Comparative synthesis: Directly contrast the three runtime representations along the axis of metadata presence and code-artifact granularity: C++ (no metadata, per-type native code), Java (shared erased representation, no metadata), C# (preserved metadata, partial code sharing).
- ENFORCE STRICT SCOPE CONTROL: confine the text strictly to runtime representation and memory/metadata structure. Do not discuss compilation times, code-bloat metrics, manual memory management, or CPU cache performance.
- Base every technical statement on authoritative references (ISO C++ Standard, JLS, JVMs, ECMA-335, and established compiler/runtime literature).

- Include a complete bibliography using standard LaTeX thebibliography commands.

Writing Style Constraint:

As defined for the instantiation-time section, with terminology adapted to runtime representation (monomorphization, name mangling, RTTI, type erasure, bridge method, reification, generic dictionary, CIL metadata).

Output Structure:

1. Introduction: Three Models of Runtime Representation
2. C++: Monomorphized Native Code and Absence of Generic Metadata
3. Java: Erased Shared Representation
4. C#: Reified CLR Representation and JIT-Constructed Types
5. Comparative Analysis: Metadata Presence and Code-Artifact Granularity
6. References

Sources:

- Vandevorde, Josuttis, Gregor, *C++ Templates: The Complete Guide* [37] – used for the monomorphization model.
- ISO/IEC 14882:2024, *Programming Language — C++* [15] – used as a normative reference.
- Stroustrup, *The C++ Programming Language* [34] – used for the C++ object model.
- Lippman, *Inside the C++ Object Model* [18] – used for the internal representation of C++ objects.
- Itanium C++ ABI Specification [16] – used for name mangling and RTTI.
- The Java Language Specification, Java SE 24 [26] – used for the erasure mechanism.
- The Java Virtual Machine Specification, Java SE 24 [27] – used for the byte-code representation.
- Bracha, *Generics in the Java Programming Language* [3] – used for bridge methods and boxing.
- ECMA-335: *Common Language Infrastructure* [11] – used for CLR reification.
- Kennedy and Syme, “Design and Implementation of Generics for the .NET CLR” [17] – used for reference-type/value-type code sharing.
- Richter, *CLR via C#* [32] – used for JIT runtime behavior.

2.4 Prompt 4: Performance Characteristics – C++, Java, and C# Compared

Goal: Generate a single, unified section analyzing the runtime performance implications of each language’s implementation strategy, following directly from the runtime representation discussion.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as a Senior Computer Science Researcher and Expert in Programming Language Theory, Compiler Design, JVM Architecture, and .NET CLR Architecture.

Goal:

Generate a comprehensive, technically rigorous academic section analyzing and directly comparing the execution performance characteristics of C++ templates (compile-time monomorphization), Java generics (type erasure), and C# generics (runtime reification with JIT specialization).

Requirements:

- Establish continuity with the following preceding theoretical context: "Both implementation strategies preserve static type safety, although they achieve this objective differently. In implementations based on type erasure, generic correctness is verified entirely during compilation. Although type parameters disappear from the generated executable, compiler-inserted casts preserve the guarantees established by the static type system. Implementations based on reified generics retain complete type information throughout compilation and generate type-specific implementations directly."
- C++: Detail the performance advantages of monomorphization: static dispatch, elimination of boxing/unboxing, and facilitation of aggressive compiler optimizations such as inlining.
- Java: Detail the runtime overheads introduced by type erasure: pointer indirection, mandatory boxing/unboxing for primitive types, impacts on heap allocation and data locality. Discuss JIT-level mitigations (escape analysis, inline caching, devirtualization) only insofar as they mitigate generic-specific overhead.
- C#: Detail the distinct performance behavior of value-type specialization (per-type native code, no boxing) versus reference-type sharing (single implementation, dictionary-passed type handles) under JIT compilation.
- Comparative synthesis: Directly compare static dispatch/AOT optimization (C++), erasure-induced boxing overhead (Java), and the hybrid value/reference-type performance profile (C#).
- ENFORCE STRICT SCOPE CONTROL: confine the analysis to execution performance, runtime throughput, memory layout, and cache behavior. Do not digress into compilation times, language ergonomics, or general garbage collection mechanics unless directly tied to generic-structure throughput.
- Rely on authoritative sources: ISO C++ Standard, JLS, JVM8, ECMA-335, C# Language Specification, and established compiler literature. Include consistent citations and a final bibliography.

Writing Style Constraint:

As defined for the instantiation-time section, with terminology adapted to performance analysis (static dispatch, boxing overhead, data locality, reification, monomorphization, type erasure, JIT specialization, cache lines).

Output Structure:

1. Introduction: Performance as a Consequence of Implementation Strategy
2. C++: Static Dispatch and Compile-Time Optimization
3. Java: Type Erasure and Boxing Overhead
4. C#: Value-Type Specialization vs. Reference-Type Sharing
5. Comparative Analysis: Throughput, Memory Layout, and Cache Behavior
6. References

Sources:

- **Vandevorde, Josuttis, Gregor, *C++ Templates: The Complete Guide* [37]** – used for the performance advantages of monomorphization.
- **Muchnick, *Advanced Compiler Design and Implementation* [25]** – used for compiler optimizations (inlining, static dispatch).
- **The Java Language Specification, Java SE 21/24 [26]** – used for boxing/unboxing behavior.
- **ECMA-335: Common Language Infrastructure [11]** – used for JIT specialization of value types.
- **Microsoft Corporation, C# Language Specification [23]** – used as a normative reference for C# generics.

2.5 Prompt 5: Type Safety – C++, Java, and C# Compared

Goal: Generate a single, unified section analyzing static type safety in C++ templates, Java generics, and C# generics, illustrated with explicative code examples for each language.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as an expert academic researcher and computer scientist specializing in programming language theory, compiler design, and cross-language generic type-system semantics.

Goal:

Generate an exhaustive, academic-grade LaTeX section analyzing static type safety across C++ templates (reified generics via monomorphization), Java generics (type erasure), and C# generics (reified generics via CLR).

Requirements:

- Integrate the following theoretical premise as the foundation: "Both implementation strategies preserve static type safety, although they achieve this objective differently. In implementations based on type erasure, generic correctness is verified entirely during compilation. Although type parameters disappear from the generated executable, compiler-inserted casts preserve the guarantees established by the static type system. Implementations based on reified generics retain complete type information throughout compilation and generate type-specific implementations directly. Since each specialization corresponds to a concrete type, correctness is preserved without

- requiring erased representations or implicit runtime casts."
- C++: Detail how templates enforce type safety at compile time via monomorphization, retaining complete type information, with a simple explanatory code example.
 - Java: Detail how type erasure verifies generic correctness entirely at compile time via compiler-inserted casts, with a simple explanatory code example.
 - C#: Detail how reified generics retain complete type information throughout compilation and generate type-specific implementations, with a simple explanatory code example.
 - Relate each code example directly to the theoretical concepts of reification, erasure, and concrete type specialization.
 - Rely on authoritative academic or technical sources (ISO/IEC 14882, JLS, ECMA-335, and peer-reviewed PLT literature).
 - ENFORCE STRICT SCOPE CONTROL: confine the text strictly to type-safety mechanisms and the theoretical strategies of implementation. Do not digress into memory footprint, hardware performance, code bloat, compilation times, or garbage collection overhead.

Writing Style Constraint:

As defined for the instantiation-time section, adapted to type safety, distinguishing theoretical concepts (parametric polymorphism) from implementation mechanisms (monomorphization, erasure, reification), with code examples formatted using the standard listings package.

Output Structure:

1. Introduction: Type Safety Under Different Implementation Strategies
2. C++: Type Safety via Reified Templates
3. Java: Type Safety via Type Erasure
4. C#: Type Safety via CLR Reification
5. Comparative Synthesis
6. References

Sources:

- **Vandevorde, Josuttis, Gregor, *C++ Templates: The Complete Guide* [37]** – used for the type-safety model of templates.
- **Stroustrup, *The C++ Programming Language* [34]** – used for static type checking in C++.
- **Pierce, *Types and Programming Languages* [29]** – used for the theoretical treatment of type safety.
- **Gosling et al., *The Java Language Specification* [12]** – used for the mechanism of compiler-inserted casts.
- **Microsoft Corporation, *C# Language Specification* [23]** – used for the reification model of C# generics.
- **ECMA-335: *Common Language Infrastructure* [11]** – used for runtime type verification in the CLR.

2.6 Prompt 6: Reflection and Runtime Type Information – C++, Java, and C# Compared

Goal: Generate a single, unified section analyzing reflection and runtime type information (RTTI) across C++ templates, Java generics, and C# generics.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as a Senior Research Scientist in Programming Language Theory, Compiler Architecture, and Type Systems, with expertise in C++ RTTI, Java reflection, and .NET System.Reflection.

Goal:

Generate a rigorous, technical-academic report section analyzing reflection and RTTI capabilities across C++, Java, and C#, grounded in the theoretical premise that runtime availability of generic type information depends directly on the chosen representation strategy: type-erased implementations discard most generic information, while reified implementations preserve it for runtime introspection.

Requirements:

- C++: Detail the application and behavior of standard RTTI mechanisms (`typeid`, `std::type_info`, `dynamic_cast`) on instantiated template classes/functions, and how monomorphization preserves concrete type information per specialization.
- Java: Detail how type erasure constrains reflection: why `instanceof` against parameterized types is disallowed, why `getClass()` yields an identical reference across structurally distinct generic instantiations, and how the type-token pattern (`Class<T>`) is used as a workaround. Include a compilation-ready Java code example.
- C#: Detail how CLR reification preserves generic metadata for full runtime introspection via `System.Reflection`, including `IsGenericType` and `GetGenericArguments()`. Include a compilation-ready C# code example.
- Comparative synthesis: Directly contrast the three reflection models along the axis of metadata availability: C++ (per-specialization RTTI, no generic-level metadata), Java (single shared `Class` object, no parameter recovery without workarounds), C# (full runtime introspection of type arguments).
- ENFORCE STRICT SCOPE CONTROL: confine the text strictly to reflection, metadata availability, and type introspection capabilities. Do not discuss memory footprint, hardware performance, code bloat, or garbage collection overhead.
- Base every technical statement on authoritative references and avoid fabricated citations.

Writing Style Constraint:

As defined for the instantiation-time section, adapted to reflection and RTTI terminology (RTTI, type erasure, reification, type token, monomorphization, generic metadata).

Output Structure:

1. Introduction: Reflection as a Function of Representation Strategy
2. C++: RTTI on Monomorphized Templates
3. Java: Type Erasure and Reflection Limitations
4. C#: Full Runtime Introspection via CLR Reification
5. Comparative Analysis
6. References

Sources:

- **Stroustrup, *The C++ Programming Language*** [34] – used for the RTTI mechanism in C++.
- **Vandevorde, Josuttis, Gregor, *C++ Templates: The Complete Guide*** [37] – used for RTTI applied to instantiated templates.
- **cppreference.com, C++ Language Reference: RTTI and typeid** [7] – used for the syntax and behavior of typeid and dynamic_cast.
- **Oracle, The Java Language Specification** [26] – used for the limitations of instanceof on parameterized types.
- **Oracle, The Java Virtual Machine Specification** [27] – used for the behavior of getClass() under erasure.
- **Richter, *CLR via C#*** [32] – used for introspection via System.Reflection in the CLR.

Chapter 3

Task 2 – Metaprogramming Across C++, Java, and C#

This chapter groups the prompts used to generate the report content addressing Task 2: *“Provide examples of metaprogramming in C++ using templates (e.g. compile-time computation, SFINAE / type traits) and compare them with what is and isn’t possible using Java and C# generics, illustrating the practical limits of metaprogramming in each ecosystem.”* The sections retain their existing individual structure – C++-native techniques are treated first, followed by dedicated sections analyzing the limits of Java and of C#, and finally the workarounds used in Java and C# to approximate C++ metaprogramming.

Note on repeated boilerplate: all prompts in this chapter share the same **Section-Level Framing** constraints (output is a single section of an existing chapter; no standalone introduction, conclusion, or bibliography; formal PLT writing style; strict scope control; enforced conciseness), stated in full in Prompt 1 and referenced thereafter rather than repeated verbatim.

3.1 Prompt 1: Concrete C++ Template Examples of Compile-Time Computation

Goal: Generate a detailed academic section of compile-time computation in C++ templates. For each example, explain what the compiler actually generates and why the computation happens entirely before runtime. Include full compilable code.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as an expert in programming language theory and advanced C++ software engineering.

Goal:

Write a single, highly focused section of a larger technical-academic chapter discussing parametric polymorphism and metaprogramming across C++, Java, and C#. This specific section must strictly focus on demonstrating and explaining C++ compile-time computation mechanisms.

Requirements:

Task 1 -- Technical Content:

- * Provide 2-3 concrete, fully compilable C++ examples of compile-time computation.
- * At least one example must demonstrate recursive compile-time computation using traditional C++ templates (e.g., calculating a factorial or Fibonacci sequence).
- * At least one example must demonstrate compile-time computation using ``constexpr`` functions.
- * For each example, provide a precise technical explanation of what the compiler actually generates during compilation.
- * For each example, explicitly detail the language mechanisms and compiler rules that guarantee the computation executes entirely before runtime.

Task 2 -- Section-Level Framing (STRICT):

- * Treat your output strictly as ONE SECTION of an existing, larger chapter.
- * Assume the reader already possesses the context of the broader comparison between C++, Java, and C\# generics/templates; do not re-introduce this context.
- * Omit any standalone introduction, motivation, or conversational framing (e.g., do not use phrases like "In this section, we will examine..."). Begin immediately with the technical exposition.
- * Omit any standalone conclusion, summary of the chapter, or forward-looking remarks.
- * Omit citations, bibliographies, or reference lists.

Task 3 -- Content Constraints:

- * ENFORCE STRICT SCOPE CONTROL: Address only the requirements stated above. Do not digress into discussions of memory footprint, hardware performance benchmarking, code bloat, garbage collection, or unrelated language comparisons.
- * Distinguish clearly between compile-time evaluation (e.g., template instantiation, constant expression evaluation) and runtime execution.
- * Use precise terminology (e.g., template instantiation, constant evaluation, monomorphization, substitution) strictly where relevant.
- * State compiler or language standard assumptions (e.g., C++11, C++14, C++20) only if strictly required to contextualize the ``constexpr`` or template mechanisms used in your examples.
- * ENFORCE CONCISENESS: The section must be as concise as possible while fully satisfying the technical requirements. Eliminate all padding, restatements of the prompt, and redundant paragraph transitions.

Writing Style Constraint:

Maintain a formal, objective, and precise technical writing style consistent with advanced programming language theory literature. Do not use first-person forms ("I", "we", "our"). Do not include explanations, comments, editorial notes, placeholders, or meta-commentary addressed to the user.

Output Structure:

Output only the raw Markdown content for this specific section. Use appropriate subsection headings (e.g., `###` or `####`) to organize the examples and explanations. Do not include a top-level title for the chapter. Return exactly the text of the section and nothing else.

Sources:

- Vandevoorde, Josuttis, Gregor, *C++ Templates: The Complete Guide* [37] – used for template instantiation and recursive compile-time metaprogramming.
- ISO/IEC 14882, C++ Language Specification [14] – used as a normative reference for template instantiation.
- [cppreference.com, documentation on constexpr](#) [8] – used for the evaluation of constant expressions at compile time.
- [cppreference.com, documentation on templates](#) [9] – used for template arguments and the specialization process.

3.2 Prompt 2: SFINAE and Type Traits

Goal: Generate a detailed academic section of a C++ example using SFINAE that selects a different function overload depending on whether a type is integral vs. floating point vs. a custom class. Also show the equivalent using C++20 concepts. Explain step by step what "substitution failure" means here and why it isn't a compile error.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as an expert technical writer and programming language theorist specialized in C++ metaprogramming and compile-time polymorphism.

Goal:

Generate a single, highly focused section for a larger academic and technical chapter on parametric polymorphism and metaprogramming. This section must exclusively demonstrate and explain C++ SFINAE and C++20 Concepts for overload resolution based on type properties.

Requirements:

* Technical Content:

- * Provide a working C++ code example using SFINAE (specifically via `std::enable_if`) that successfully selects a different function overload depending on whether a type is an integral type, a floating-point type, or a custom class.
- * Provide the equivalent working C++ code example utilizing C++20 Concepts.
- * Provide a step-by-step explanation of what "substitution failure" means in this specific context and explicitly detail the mechanism of why it does not result in a hard compile error.

* Content Constraints:

- * Distinguish clearly that these are compile-time mechanisms.

* Enforce strict scope control: address only the SFINAE implementation, the C++20 Concepts equivalent, and the explanation of substitution failure. Do not digress into compilation times, memory footprint, code bloat, hardware performance, or other unrelated language comparisons.

Output Structure:

Use standard Markdown code blocks for the C++ code examples.

Sources:

- **Stroustrup, *The C++ Programming Language*** [34] – used for compile-time polymorphism based on templates.
- **cppreference.com** [7] – used for the behavior of `std::enable_if` and C++20 Concepts syntax.
- **ISO/IEC 14882:2020** [14] – used as a normative reference for SFINAE rules and the C++20 constraint system.

3.3 Prompt 3: Variadic Templates, Template Template Parameters, and CRTP

Goal: Generate a detailed academic section of other core C++ template metaprogramming techniques: variadic templates, template template parameters, and CRTP (Curiously Recurring Template Pattern). For each, give a short example and explain what class of problems it solves that plain function/class templates cannot.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as a technical-academic writer specialized in programming language theory and advanced C++ software engineering.

Goal:

Write a single, highly focused section for a larger chapter on parametric polymorphism and metaprogramming. The section must explain three specific advanced C++ template metaprogramming techniques: variadic templates, template template parameters, and the Curiously Recurring Template Pattern (CRTP).

Requirements:

- * **Technical Scope:** Focus exclusively on variadic templates, template template parameters, and CRTP. Explicitly exclude discussions of SFINAE, type traits, and `constexpr`, as these are assumed to be covered elsewhere.
- * **Content Execution:** For each of the three covered techniques:
- * Provide a short, syntactically correct C++ code example.
- * Explain the specific class of problems the technique solves that plain function or class templates cannot resolve.
- * Clearly distinguish compile-time mechanisms and state relevant assumptions regarding C++ language/compiler versions (e.g., C++11 for variadic templates) where necessary.

* Strict Scope Control: Do not digress into memory footprint, code bloat, hardware performance benchmarking, or unrelated language comparisons. Address only the mechanisms requested.

Sources:

- Vandevorde, Josuttis, Gregor, *C++ Templates: The Complete Guide* [37] – used for variadic templates, template template parameters, and CRTP.
- cppreference.com [7] – used for C++ construct syntax and template specialization.
- Microsoft, *C++ templates documentation* [20] – used as a complementary reference for practical syntax examples.

3.4 Prompt 4: Limits of Java Generics in Replicating C++ SFINAE and Type Traits

Goal: Generate a detailed academic section on the limits of Java generics in replicating C++ SFINAE and type traits. Show what is achievable (e.g. bounded type parameters like `<T extends Number>`) and explain precisely why full SFINAE-style compile-time branching on the generic type `T` is impossible in Java, tying the explanation to type erasure.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as an expert in programming language theory and software engineering, specializing in the comparative analysis of parametric polymorphism and metaprogramming across C++, Java, and C\#.

Goal:

Write a single, concise section of an academic report chapter evaluating the limits of Java generics when attempting to replicate C++ SFINAE and type-traits behavior. Specifically, demonstrate the extent of what is achievable in Java (e.g., bounded type parameters like `<T Number extends>`) and explain precisely why full SFINAE-style compile-time branching on a generic type `T` is impossible in Java, explicitly tying this architectural limitation to Java's implementation of type erasure.

Requirements:

- * Technical Demonstration: Provide a concise, valid Java code example demonstrating the maximum achievable compile-time constraint mechanism in Java, specifically bounded polymorphism (e.g., `<T Number extends>`).
- * Theoretical Explanation: Detail exactly why full compile-time branching on generic types (the equivalent of C++ Substitution Failure Is Not An Error) fails in Java. Connect this limitation mechanically and theoretically to type erasure.
- * Terminology: Use precise technical terminology appropriate for programming language theory (e.g., type erasure, bounded polymorphism, monomorphization, compile-time resolution, bounds checking) where directly relevant.

* Scope Control: Address only the replication of SFINAE/type-traits in Java. Do not digress into memory footprint, hardware performance, C\# mechanisms, code bloat, garbage collection, or unrelated language comparisons.

Sources:

- Bracha et al., “Making the Future Safe for the Past” [4] – used for the type erasure model and the limitations of parametric polymorphism in Java.
- Gosling et al., *The Java Language Specification* [12] – used for the semantics of the extends constraint and the erasure process.
- Vandevoorde, Josuttis, Gregor, *C++ Templates: The Complete Guide* [37] – used as a comparison for SFINAE substitution and type traits in C++.

3.5 Prompt 5: Limits of C# Generics in Replicating C++ SFINAE and Type Traits

Goal: Generate a detailed academic section on the limits of C# generics in replicating C++ SFINAE and type traits. Show what’s achievable using generic constraints (`where T : ...`) and, if relevant, .NET generic math / static abstract interface members. Explain why C# gets closer to C++ than Java does, but still can’t do arbitrary compile-time computation or true SFINAE, tying it to reification vs. monomorphization.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as an expert academic writer and programming language theorist specializing in the comparative analysis of parametric polymorphism and metaprogramming paradigms.

Goal:

Generate a single, concise, and highly technical section for a larger academic chapter. This section must specifically analyze how C# approximates C++ SFINAE and type-traits using generic constraints and generic math, while explaining its theoretical limitations.

Requirements:

- * Technical Execution:
 - * Replicate the conceptual equivalent of a C++ SFINAE/type-traits example using C#.
 - * Demonstrate achievable compile-time/type-check-time constraints using C# bounded polymorphism (``where T : ...``).
 - * Illustrate the application of .NET generic math and static abstract interface members to achieve trait-like capabilities.
- * Theoretical Analysis:
 - * Explain why these C# mechanisms allow it to approach C++ capabilities much more closely than Java does.
 - * Explain why C# inherently cannot support true SFINAE (Substitution Failure Is Not An Error) or arbitrary compile-time computation.

```
* Directly tie this limitation to the architectural differences between
.NET's runtime reification of generics and C++'s compile-time
monomorphization and AST-level substitution.

* Scope Control: address solely the requested C\# replication of
SFINAE/traits, generic constraints, generic math, and the reification vs.
monomorphization dichotomy. Do not digress into memory footprint, code bloat,
execution performance, garbage collection, or unrelated language features.
```

Sources:

- **Microsoft, C# generics documentation** [22] – used for generic constraints and compile-time validation of the `where` clause.
- **Microsoft, .NET Generic Math documentation** [21] – used for static abstract interface members and interfaces such as `INumber<T>`.
- **Vandevoorde, Josuttis, Gregor, *C++ Templates: The Complete Guide*** [37] – used as a comparison with SFINAE, type traits, and C++ monomorphization.
- **ECMA-335: Common Language Infrastructure** [11] – used for the CLR execution model and generics reification.

3.6 Prompt 6: Instantiation-Time vs. Declaration-Time Checking

Goal: Generate a detailed academic section explaining how the difference between C++ templates (checked only at instantiation) and Java/C# generics (checked at declaration) directly causes specific metaprogramming techniques to be impossible in Java/C#.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as a technical author and programming language theorist writing a highly specialized section for an academic chapter on programming language theory and software engineering.

Goal:

Generate a single, self-contained section of a larger chapter that explains how the difference between C++ templates (checked only at instantiation) and Java/C# generics (checked at declaration) directly causes specific metaprogramming techniques to be impossible in Java/C#. You must detail at least one concrete technique that fails specifically due to this difference.

Requirements:

- * Focus strictly on the mechanisms of type checking: instantiation-time (C++) versus declaration-time (Java/C#).
- * Explain the direct causal relationship between these checking mechanisms and the limitations placed on metaprogramming in Java and C#.
- * Detail at least one concrete metaprogramming technique (e.g., SFINAE, or structural/duck typing within templates) that is possible in C++ but impossible in Java/C# because of this specific constraint.

- * Include concise, precise code examples to illustrate the technique in C++ and explicitly demonstrate why declaration-time checking prevents its equivalent in Java/C\#.
- * Use precise terminology (e.g., instantiation, substitution failure, bounded polymorphism, type erasure, reification) where strictly relevant to the prompt.
- * Clearly distinguish between compile-time and runtime mechanisms as they relate to these type-checking models.

Sources:

- Vandevorde, Josuttis, Gregor, *C++ Templates: The Complete Guide* [37] – used for the delayed type checking of templates and SFINAE.
- Bracha, *Generics in the Java Programming Language* [3] – used for the declaration-time checking model of Java generics.
- ECMA-335: *Common Language Infrastructure* [11] – used for the compile-time verification rules of C\# generics.

3.7 Prompt 7: External Metaprogramming Workarounds in Java and C\#

Goal: Generate a detailed academic section on the external metaprogramming workarounds in Java and C\#. Explain why these are workarounds external to the generics system rather than native metaprogramming.

Model: ChatGPT-5.5

Prompt:

System Persona:

Act as an academic technical writer specializing in programming language theory and software architecture.

Goal:

Write exactly ONE SECTION of an existing comprehensive chapter on parametric polymorphism and metaprogramming. This specific section must describe the common workarounds used in Java and C\# to simulate the native metaprogramming capabilities of C++ templates, and architecturally explain why these approaches are external to their respective generics systems rather than native metaprogramming.

Requirements:

- * Technical Content Specification:
- * Detail Java's common workarounds: the use of runtime reflection, compile-time annotation processors, and code generation utilities (e.g., Lombok).
- * Detail C\#'s common workarounds: the use of source generators and T4 templates.
- * Articulate the structural and theoretical reasons why these mechanisms are considered external workarounds layered over or completely separate from the Java and C\# generics systems, contrasting them with the native, intrinsic compile-time evaluation of C++ templates.

```
* Clearly differentiate between mechanisms that operate at runtime versus
those that operate at compile-time or pre-compilation.

* Content Constraints:
* Use accurate terminology (e.g., compile-time, runtime, reflection, AST
manipulation, macro expansion) strictly where relevant.
* Include brief, illustrative code snippets only if strictly necessary to
demonstrate a specific structural workaround mechanism; otherwise, rely on
rigorous theoretical explanation.
* Strictly enforce scope control: address only the workarounds and their
relationship to the generics system. Do not digress into memory footprint,
hardware performance benchmarking, garbage collection overhead, or unrelated
architectural comparisons.
```

Sources:

- **Oracle, Java compiler API documentation** [28] – used for annotation processing and reflection as a runtime mechanism.
- **Lombok Project, official documentation** [31] – used for code generation via compiler extensions.
- **Microsoft, C# code generation documentation** [24] – used for Roslyn-based Source Generators and T4 Templates.
- **Vandevorde, Josuttis, Gregor, *C++ Templates: The Complete Guide*** [37] – used for the template instantiation model as native metaprogramming.

Bibliography

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson, 3 edition, 2022.
- [2] Andrew W. Appel. *Modern Compiler Implementation in C*. Cambridge University Press, 1998.
- [3] Gilad Bracha. Generics in the java programming language, 2004.
- [4] Gilad Bracha, Martin Odersky, David Stoutamire, and Philip Wadler. Making the future safe for the past: Adding genericity to the java programming language. In *Proceedings of the 13th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 183–200. ACM, 1998.
- [5] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.
- [6] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, 3 edition, 2022.
- [7] cppreference.com. C++ language reference: Rtti and typeid, 2025. Online reference for C++ RTTI mechanisms, std::type_info, typeid, and dynamic_cast.
- [8] cppreference.com. constexpr specifier. <https://en.cppreference.com/w/cpp/language/constexpr>, 2025.
- [9] cppreference.com. Templates. <https://en.cppreference.com/w/cpp/language/templates>, 2025.
- [10] Ecma International. Ecma-334: C# language specification, 2023.
- [11] Ecma International. Ecma-335: Common language infrastructure (cli), 2024.
- [12] James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley. *The Java Language Specification, Java SE 8 Edition*. Oracle, 2014.
- [13] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight java: A minimal core calculus for java and gj. *ACM Transactions on Programming Languages and Systems*, 23(3):396–450, 2001.
- [14] ISO/IEC. *ISO/IEC 14882:2020 Programming Languages – C++*. International Organization for Standardization, 2020.
- [15] ISO/IEC JTC 1/SC 22/WG 21. *ISO/IEC 14882:2024 Programming Languages — C++*, 2024. International Standard.
- [16] Itanium C++ ABI Project. Itanium c++ abi: C++ abi specification, 2024. C++ Application Binary Interface specification including name mangling and RTTI representation.
- [17] Andrew Kennedy and Don Syme. Design and implementation of generics for the

- .net common language runtime. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 1–12. ACM, 2001.
- [18] Stanley B. Lippman. *Inside the C++ Object Model*. Addison-Wesley Professional, Reading, MA, 1996.
 - [19] John McCall. Generics in swift: Implementation and runtime representation, 2019.
 - [20] Microsoft. *C++ Templates Documentation*. Microsoft Learn, 2024.
 - [21] Microsoft. Generic math in .net. <https://learn.microsoft.com/en-us/dotnet/standard/generics/math>, 2025.
 - [22] Microsoft. Generics in c#. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/generics>, 2025.
 - [23] Microsoft Corporation. *C# Language Specification*. Microsoft, 2023.
 - [24] Microsoft Corporation. Code generation in c#: Source generators and t4 text templates. <https://learn.microsoft.com/dotnet/csharp/roslyn-sdk/source-generators-overview>, 2024.
 - [25] Steven S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
 - [26] Oracle. The java language specification, java se 24 edition, 2024.
 - [27] Oracle. The java virtual machine specification, java se 24 edition, 2024.
 - [28] Oracle Corporation. Java compiler api and annotation processing. <https://docs.oracle.com/javase/>, 2024.
 - [29] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.
 - [30] Benjamin C. Pierce, editor. *Advanced Topics in Types and Programming Languages*. MIT Press, Cambridge, MA, 2005.
 - [31] Project Lombok. Project lombok: Spice up your java. <https://projectlombok.org/>, 2024.
 - [32] Jeffrey Richter. *CLR via C#: A Developer’s Tour of the Common Language Runtime*. Microsoft Press, 4 edition, 2012.
 - [33] Christopher Strachey. Fundamental concepts in programming languages, 1967. Lecture Notes, International Summer School in Computer Programming, Copenhagen.
 - [34] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley Professional, 4 edition, 2013.
 - [35] The Rust Project Developers. Monomorphization and code generation, 2024.
 - [36] The Rust Project Developers. The rust reference, 2024. Official Rust language specification.
 - [37] David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2 edition, 2017.